Introduction to

Information Retrieval

Spelling Correction

Spelling Correction

- Two principal uses:
  - Correcting document(s) being indexed
  - Correcting user queries to retrieve "right" answers
- Two main flavors:
  - Isolated word
    - Check each word on its own for misspelling
    - Will not catch typos resulting in correctly spelled words
    - e.g., from → form
  - Context-sensitive
    - Look at surrounding words
    - e.g., I flew form Heathrow to Narita.

Document Correction

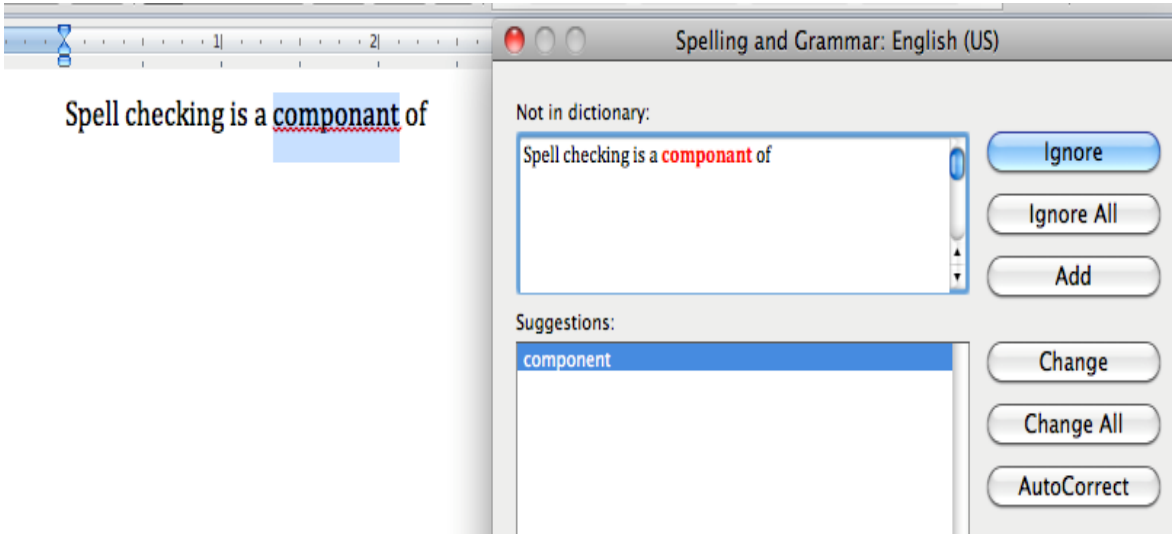
- Especially needed for OCR documents
  - Correction algorithms are tuned for this: rn/m
  - Can use domain-specific knowledge
    - E.g., OCR can confuse O and D more often than it would confuse O and I (adjacent on the QWERTY keyboard, so more likely interchanged in typing)
- But also: web pages and even printed material have typos
- Goal: the dictionary contains fewer misspellings
  - Reduced vocabulary size
  - Include just English words?
- But often we don't change the documents and instead fix the query-document mapping

Query mis-spellings

- Our principal focus here
  - E.g., the misspelled query *Alanis Morisett*
- We can either:
  - Retrieve documents indexed by the correct spelling, OR
  - Return several suggested alternative queries with the correct spelling
    - *Did you mean ...?*
    - Search instead for …

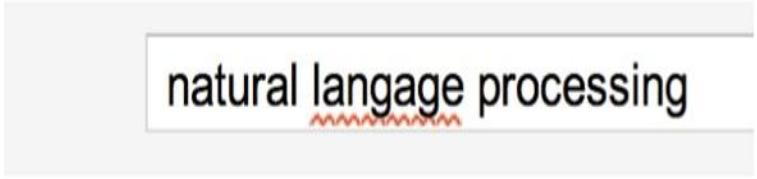
Applications for spelling correction

Word processing



Phones

Web search



Showing results for natural ***language*** processing

Search instead for natural langage processing

Isolated word correction

- Fundamental premise – there is a lexicon from which the correct spellings come
- Two basic choices for this:
  - A standard lexicon such as:
    - Webster's English Dictionary
    - An "industry-specific" lexicon – hand-maintained
  - The lexicon of the indexed corpus
    - E.g., all words on the web
    - All names, acronyms etc.
    - (Including the mis-spellings)

Isolated word correction

- Given a lexicon and a character sequence Q, return the words in the lexicon closest to Q
- What's "closest"?
- We'll study several alternatives:
  - Edit distance
    - Levenshtein distance
  - Weighted edit distance
  - n-gram overlap

Edit distance

- Given two strings S1 and S2, the minimum number of operations to convert one to the other
- Operations are typically character-level
  - Insert, Delete, Replace, (Transposition)
- E.g., the edit distance from **do**f to **dog** is 1
  - From **cat** to **act** is 2
    (Just 1 with transpose.)
  - From **cat** to **dog** is 3.
- Generally found by dynamic programming.

Weighted edit distance

- As above, but the weight of an operation depends on the character(s) involved
  - Meant to capture OCR or keyboard errors
    Example: m more likely to be mis-typed as **n** than as **q**
  - Therefore, replacing **m** by **n** is a smaller edit distance than by **q**
  - This may be formulated as a probability model
- Requires weight matrix as input
- Modify dynamic programming to handle weights

Using edit distances

- Given query, first enumerate all character sequences within a preset (weighted) edit distance (e.g., 2)
- Intersect this set with list of "correct" words
- Show terms you found to user as suggestions
- Alternatively:
  - We can look up all possible corrections in our inverted index and return all docs ... slow
  - We can run with a single most likely correction
- The alternatives disempower the user, but save a round of interaction with the user

Edit distance to all dictionary terms?

- Given a (misspelled) query – do we compute its edit distance to every dictionary term?
  - Expensive and slow
  - Alternative?
- How do we cut the set of candidate dictionary terms?
- One possibility is to use n-gram overlap for this.
- This can also be used by itself for spelling correction.

n-gram overlap

- Enumerate all the n-grams in the query string as well as in the lexicon
- Use the n-gram index to retrieve all lexicon terms matching any of the query n-grams
- Threshold by number of matching n-grams
  - variants – weight by keyboard layout, etc.

Example with trigrams

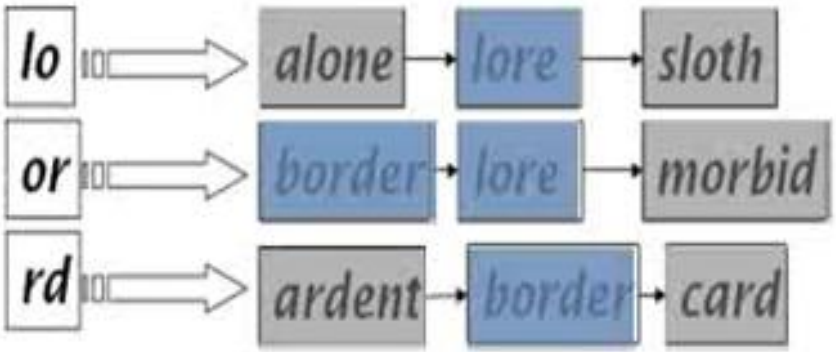
- Suppose the text is **november**
  - Trigrams are nov, ove, vem, emb, mbe, ber
- The query is **december**
  - Trigrams are dec, ece, cem, emb, mbe, ber
- So 3 trigrams overlap (of 6 in each term)
- How can we turn this into a normalized measure of overlap?

One option – Jaccard coefficient

- A commonly-used measure of overlap
- Let X and Y be two sets; then the J.C. is
- $|X \cap Y|/|X \cup Y|$ Equals 1 when X and Y have the same elements and zero when they are disjoint
- X and Y don't have to be of the same size
- Always assigns a number between 0 and 1
  - Now threshold to decide if you have a match
  - E.g., if J.C. > 0.8, declare a match

Matching Trigrams

- Consider the query ***lord*** – we wish to identify words matching 2 of its 3 bigrams (***lo, or, rd***)



Standard postings "merge" will enumerate ...

Adapt this to using Jaccard (or another) measure.

Context-sensitive spell correction

- Text: I flew from Heathrow to Narita.
- Consider the phrase query "flew form Heathrow"
- We'd like to respond
- Did you mean "flew from Heathrow"?
- because no docs matched the query phrase.

Context-sensitive spell correction

- Need surrounding context to catch this.
- First idea: retrieve dictionary terms close (in weighted edit distance) to each query term
- Now try all possible resulting phrases with one word "fixed" at a time
- flew from heathrow
- fled form heathrow
- flea form heathrow
- Hit-based spelling correction: Suggest the alternative that has lots of hits.

Exercise

- Suppose that for “flew form Heathrow” we have 7 alternatives for flew, 19 for form and 3 for heathrow.

- How many “corrected” phrases will we enumerate in this scheme?